Cost-Efficient Segmentation under a Compute Budget

CSL7360 Computer Vision — Assignment 2 (Track B)

Jatin Purbia (B23CH1022)
Indian Institute of Technology Jodhpur

2 November 2026

Parameters	File size	CPU time (DUTS-TE)	DUTS-TE Q	DUTS-TE CES
439,393	1.79 MB	17.1 ms (limit 50)	0.633	0.563

Abstract. We solve Track B of the assignment with a small U-Net trained from scratch (439,393 parameters, 1.79 MB, no pretrained weights) that labels every pixel of a 256×256 photo as object or background. The network is first trained with BCE and then fine-tuned for six epochs with BCE+Dice and mild augmentation; it runs internally at 128×128 with a channels-last tensor layout, which keeps the mean single-thread CPU time near 17 ms against the 50 ms budget. All choices were made on a held-out part of DUTS-TR. On the untouched DUTS-TE set the model reaches mIoU 0.714, BF1 0.444, $Q = 0.633$ and CES $= 0.563$. On our ten own phone photographs, scored against the supplied masks, it reaches mIoU 0.623, BF1 0.324, $Q = 0.533$ and CES $= 0.480$.

1 Introduction

The task is binary salient-object segmentation: assign each pixel to the main object(s) or background. The objective is to maximize the Cost-Efficient Segmentation (CES) score subject to a mean single-thread CPU time of at most 50 milliseconds per photo. Region overlap alone is insufficient: the quality term also rewards accurate boundaries, and CES subtracts a small charge for time and file size. We follow Track B (a fully convolutional network). All code uses only the Python standard library, PyTorch, NumPy, OpenCV and Matplotlib, and the metrics are written by hand.

2 Dataset and leakage prevention

DUTS contains 10,553 training photographs (DUTS-TR) and 5,019 test photographs (DUTS-TE) [2]. A sorted list of DUTS-TR stems is shuffled using Python's seed-42 random generator; round($0.1N$) IDs form validation. For the official training size this gives 9,498 training and 1,055 validation IDs. The actual manifests, once prepared, are saved and hashed. Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42.

All architecture, loss, threshold, resolution and stopping decisions use only DUTS-TR validation. The experiment runner never loads DUTS-TE. Final evaluation requires a frozen parameter-file hash and records all 5,019 samples individually. Personal photos are testing-only and require 5–10 authentic phone captures covering at least three condition categories. A script can validate files and author attestations; it cannot prove photographic authenticity or scene diversity.

3 Preprocessing

OpenCV reads BGR uint8 photographs. Every original photo is directly resized to 256×256 using `INTER_AREA`; aspect ratio is intentionally squashed. Grayscale masks use `INTER_NEAREST` and are binarized at intensity 128 inclusive. The network receives RGB float32 tensors $[B, 3, H, W]$ in $[0, 1]$, with no mean/std normalization. Training augmentation jointly flips image/mask horizontally with probability 0.5 and applies brightness/contrast multipliers sampled uniformly in $[0.9, 1.1]$ to images only. Validation and test preprocessing are deterministic.

The timed pipeline starts after the original resize. It includes BGR-to-RGB conversion, tensor creation, normalization, optional bilinear downscaling, forward inference, bilinear upsampling of logits to 256×256, sigmoid, thresholding and conversion to a binary uint8 array. Masks on disk are multiplied by 255 after timing.

4 Proposed architectures

All candidates are encoder–decoder networks built from convolution, batch normalization, ReLU, max pooling, bilinear upsampling and concatenation skips. Three families were explored in short pilot runs (Tiny FCN: one convolution per stage; U-Net: two per stage; depthwise-separable). The selected model is the U-Net with base width $b = 16$ and four stages: encoder widths $16, 32, 64, 128$, each stage with two Conv–BN–ReLU blocks, 2×2 max pooling between stages, then three decoder stages that bilinearly upsample, concatenate the matching encoder feature map and apply Conv–BN–ReLU. A 1×1 convolution produces one logit per pixel.

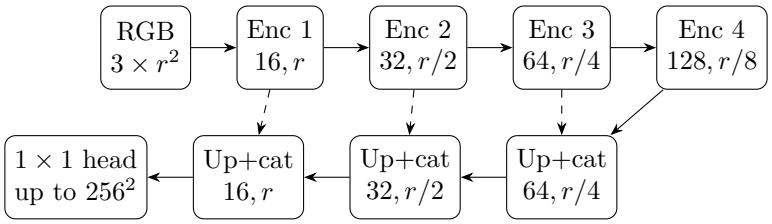


Figure 1. Selected U-Net ($r = 128$ at inference and in training). Dashed arrows are concatenation skips.

Selected release: fine-tuned U-Net (stage 2). Architecture: {kind: unet, base 16, depth 4}; learned parameters: 439393; inference resolution: 128 (channels-last); threshold: 0.5.

5 Mathematical formulation

For input channel c, output channel o and spatial offsets (u, v), convolution is

$$z_{oij} = a_o + \sum_c \sum_u \sum_v W_{ocuv} x_{c,i+u,j+v}. \tag{1}$$

Here W contains learned filters and a is a bias (omitted in normalized blocks). For logit z_i, probability p_i and binary label y_i,

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \qquad \hat{y}_i = \mathbf{1}[p_i \geq \tau], \tag{2}$$

$$L_{\text{BCE}} = -\frac{1}{N} \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right], \tag{3}$$

$$L_{\text{Dice}} = 1 - \frac{2 \sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + \epsilon}, \qquad \epsilon = 10^{-6}, \tag{4}$$

$$L = L_{\text{BCE}} + \beta L_{\text{Dice}}. \tag{5}$$

BCE is computed through the stable logits formulation. Dice is computed separately for each image and averaged. The default decision threshold is $\tau = 0.5$; any change must use validation only. The mandatory baseline has $\beta = 0$.

Aggregate all pixels in the evaluated dataset into

$$C = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}, \qquad TP = \sum_i \mathbf{1}[\hat{y}_i = 1, y_i = 1], \tag{6}$$

with analogous definitions for true negatives, false positives and false negatives. Rows denote true class; columns denote predicted class. Then

$$\mathrm{IoU}_{fg} = \frac{TP}{TP + FP + FN}, \qquad \mathrm{IoU}_{bg} = \frac{TN}{TN + FP + FN}, \tag{7}$$

$$\mathrm{mIoU} = \tfrac{1}{2}(\mathrm{IoU}_{fg} + \mathrm{IoU}_{bg}). \tag{8}$$

We assign an absent class (zero union) IoU of 1, representing correct absence. This is an explicit implementation convention because the PDF does not define that degeneracy. Per-image IoUs are never averaged to form the dataset score.

For foreground set M and four-connected neighborhood $\mathcal{N}_4$,

$$\partial M = \{i \in M : \exists j \in \mathcal{N}_4(i),\ j \notin M\}. \tag{9}$$

Pixels outside the image are background. Let $B_p = \partial \hat{M}$, $B_t = \partial M$, and $D_2(B)$ be pixels within a centered 5×5 window of B. Matching tolerance is Chebyshev distance two, including diagonal offsets $(2, 2)$. For one image,

$$P = \frac{|B_p \cap D_2(B_t)|}{|B_p|}, \qquad R = \frac{|B_t \cap D_2(B_p)|}{|B_t|}, \tag{10}$$

$$F_1 = \begin{cases} 2PR/(P+R) & P + R > 0, \\ 0 & \text{otherwise}, \end{cases} \qquad BF1 = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}. \tag{11}$$

Both empty boundaries yield $F_1 = 1$; exactly one empty boundary yields $F_1 = 0$. Four directional tensor slices extract the boundary. Neighborhood matching uses 25 whole-array boolean OR passes over zero-padded NumPy views, with no per-pixel Python loop or library morphology. Analytic corner/diagonal tests and equivalence checks against the permitted `max_pool2d` building block verify the exact window. Matching is many-to-many, as specified, rather than bipartite assignment. This optimization accelerates evaluation; metrics remain outside the inference timer.

For all quality metrics on the $[0, 1]$ scale,

$$Q = 0.70\,\mathrm{mIoU} + 0.30 BF1, \tag{12}$$

$$CES = \begin{cases} Q\left[1 - 0.20\frac{T_{CPU}}{50} - 0.10\min\left(1, \frac{S_{disk}}{4194304}\right)\right], & T_{CPU} \le 50, \\ 0, & T_{CPU} > 50. \end{cases} \tag{13}$$

T_{CPU} is in milliseconds and S_{disk} is the actual parameter-file size in bytes. For $Q = .7$, $T = 20$ and $S = 524288$, the exact result is .63525, consistent with the PDF's rounded .635.

6 Training methodology

Stage 1 (scratch). The network was trained once for 16 epochs with AdamW (learning rate 3×10^{-3}, weight decay 10^{-4}), cosine learning-rate decay, batch size 16, gradient-norm clipping at

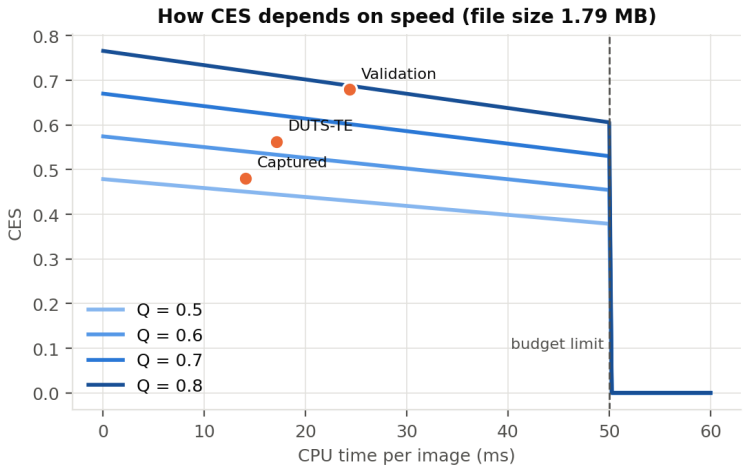


Figure 2. CES as a function of CPU time for four quality levels. The penalty grows linearly with time, so halving the CPU time raises CES by only a few percent, whereas a gain of 0.1 in Q raises it by about 0.09. The three orange points are our three evaluation sets.

5 and pure BCE loss ($\beta = 0$), using the 9,498 training images at 128×128 internal resolution; the loss is computed on logits upsampled to 256×256. The best epoch by validation Q (epoch 15, $Q = 0.7877$ at $r = 128$) was kept. Training took about 10 minutes per epoch on the CPU listed below using 10 threads; the timed pipeline always uses one. Validation Q rose steadily from 0.51 (epoch 1) to 0.79 (epoch 16), see Figure 3.

Stage 2 (fine-tuning). Starting from the best stage-1 weights, we continued for six epochs with a fresh AdamW optimizer (learning rate 3×10^{-4}, cosine decay), loss $L_{\text{BCE}} + 0.3\, L_{\text{Dice}}$, and mild paired augmentation: random scale ($\pm 15\%$) and translation ($\pm 10\%$) applied jointly to image and mask with probability 0.5, plus brightness/contrast ($\pm 20\%$) and saturation ($\pm 15\%$) changes on the image only. The validation split is unchanged; neither DUTS-TE nor the captured photographs enter training or selection. Epoch 6 was best ($Q = 0.7899$, Figure 4). This is a combined improvement, not an isolated ablation of each change. The released weights had to pass validation gates against the stage-1 model (gain in Q of at least 0.005, no loss of BF1, at most 0.002 loss of mIoU, mean CPU at most 40 ms, p95 CPU at most 50 ms, a positive paired bootstrap interval, and no loss under dark, bright and low-contrast stress); all gates passed, with a paired 95% interval for the validation Q gain of [0.011, 0.021]. Each run stores its configuration, package versions, split hashes and per-epoch log.

7 Computational efficiency and timing

The encoder operates at a configurable internal resolution; the full output remains 256×256. Standard blocks prioritize simple CPU kernels, while separable blocks reduce arithmetic at the possible cost of additional operator overhead. The selected model stores tensors in PyTorch's channels-last memory format, a standard CPU layout option (not another inference engine) whose conversions are included in the timed function. Single-thread execution is configured before NumPy, OpenCV or PyTorch imports through four required environment variables, followed by their thread-setting APIs. Inference uses CPU tensors, `eval()` and `inference_mode()`. No inference subprocess, data-loader worker or alternate engine is used.

One full warmup is excluded. For official timing, each DUTS-TE photo is loaded and resized outside the timer. The exact shared inference function is wrapped by `time.process_time()`. The sum of all 5,019 durations divided by 5,019 gives T_{CPU}. Metrics and file exports occur

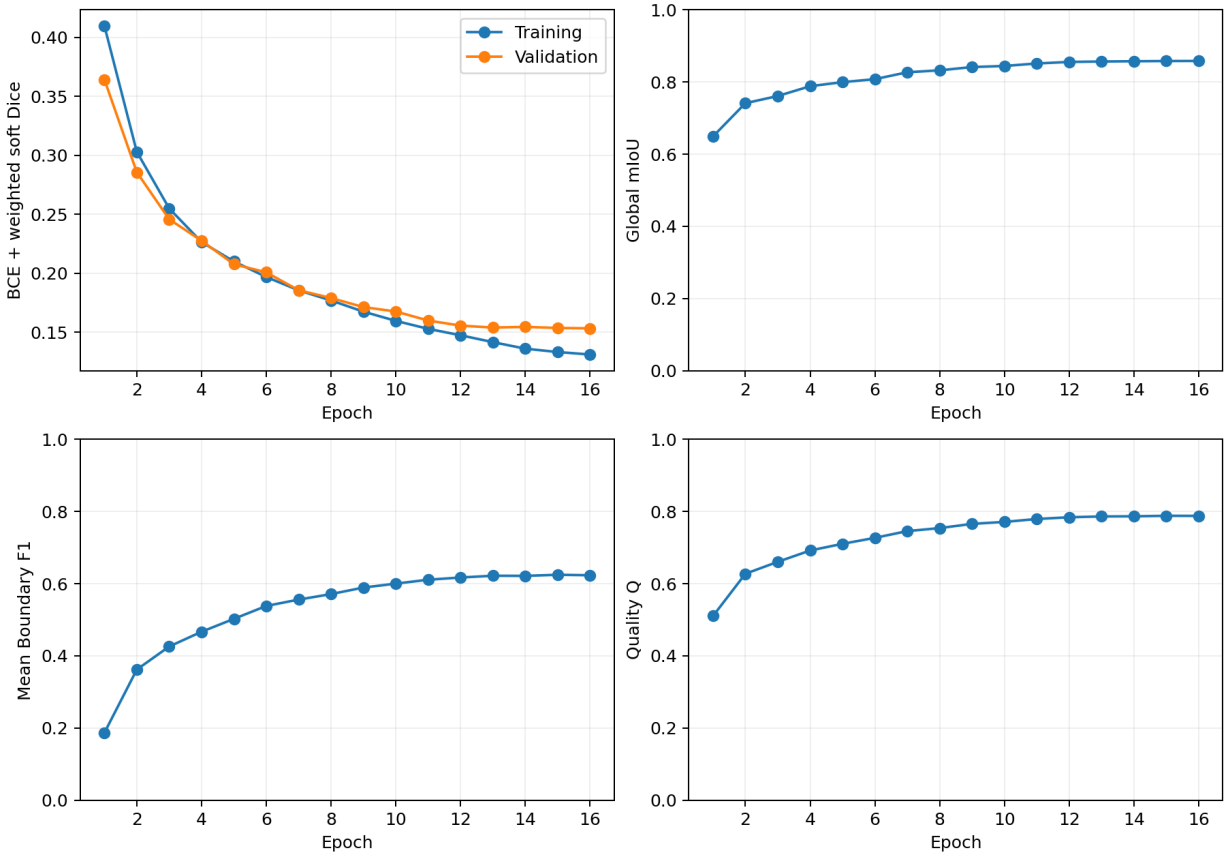


Figure 3. Stage 1: training and validation curves of the scratch run.

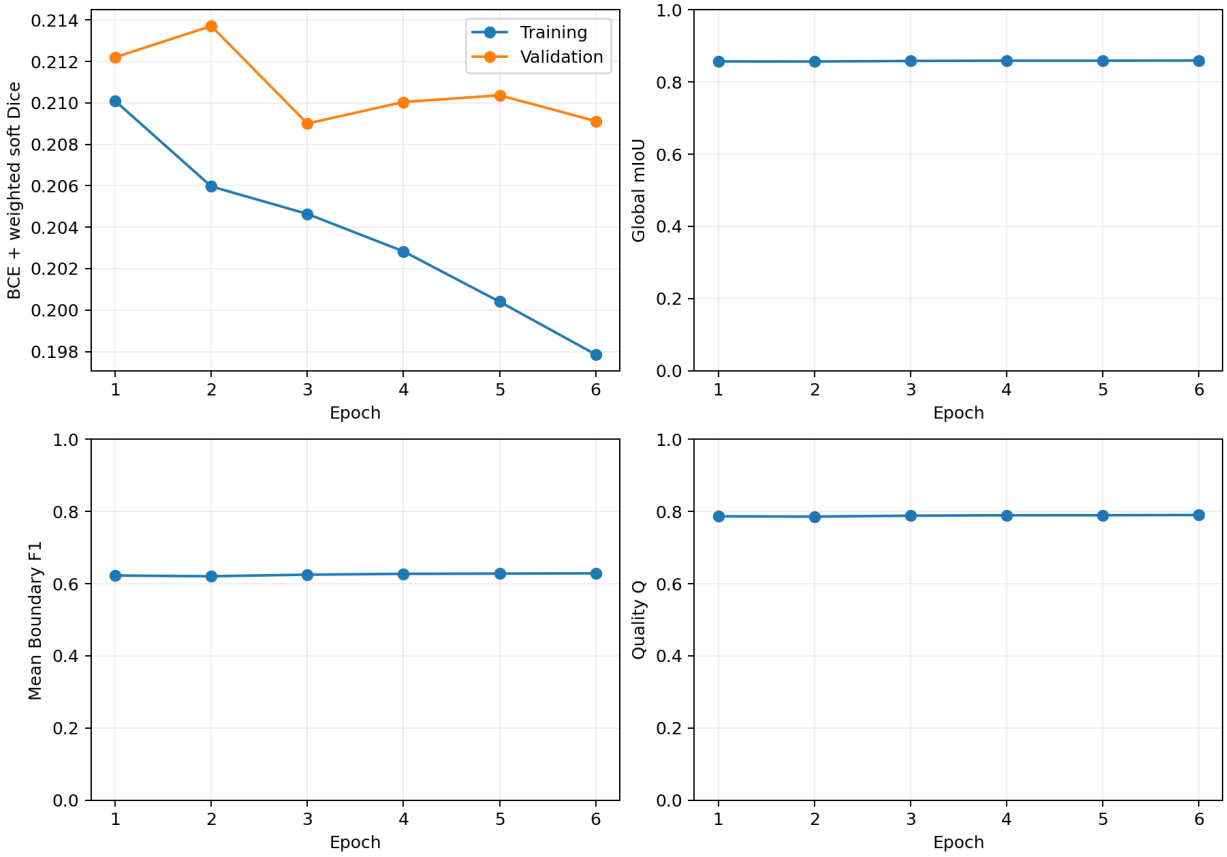


Figure 4. Stage 2: curves of the BCE+Dice fine-tuning run (selected release).

outside timing. The prediction CLI instead uses its output-producing pass as warmup, then averages 20 additional full-pipeline runs. Size uses `os.path.getsize`; plain `torch.save` is not post-compressed.

8 Experimental comparison

Before the final run we ran three two-epoch pilots (Tiny FCN with BCE, Tiny FCN with BCE+Dice, depthwise-separable FCN with BCE; about 3–4k parameters each). They reached validation Q between 0.45 and 0.47, with BCE and BCE+Dice indistinguishable at this budget and the depthwise variant slower on CPU. We therefore moved to the larger U-Net.

On the trained stage-1 U-Net we varied the inference resolution and memory layout on the validation set. Channels-last cut the 128-pixel time from 33.2 to 21.6 ms with identical Q. The fine-tuned model was then evaluated at four resolutions (Table 1). Because CES charges $0.2\,T/50$ for time, the speed-focused row (fine-tuned, $r = 96$) has the highest validation CES (0.700). The release rule nevertheless prioritizes quality among candidates that pass the budget gates, so the fine-tuned $r = 128$, $\tau = 0.5$ candidate with $Q = 0.7899$ was chosen; its validation CES (0.679) is lower but its mean test time is still only 17 ms. Resolutions above the training size lose quality. Per-row validation timings are noisy by a few ms because other programs ran concurrently; the timing in Table 2 for DUTS-TE comes from a clean 5,019-image measurement. Rows come from `results/quality/*.validation.json`.

Model	Setting	mIoU	BF1	Q	Val. ms	CES
Stage 1	$r = 96$	0.8503	0.5958	0.7739	22.4	0.6716
Stage 1	$r = 128$, ch.-last	0.8576	0.6245	0.7877	21.6	0.6862
Fine-tuned	$r = 96$	0.8538	0.6016	0.7782	14.4	0.7002
Fine-tuned	$r = 112$	0.8595	0.6205	0.7878	20.5	0.6896
Fine-tuned	$r = 128$ (selected)	0.8591	0.6284	0.7899	24.3	0.6794
Fine-tuned	$r = 144$	0.8514	0.6191	0.7817	27.7	0.6619

Table 1. Validation-only comparison (threshold 0.5, channels-last except the first row). Timings are from one noisy session.

Figure 5 shows why the final run moved to a larger network. The tiny and depthwise networks (3–4k parameters) stay at $Q \approx 0.46$ whatever the resolution, because they lack the capacity to separate objects from clutter; spending more time on them buys nothing. The 439k-parameter U-Net reaches $Q \approx 0.79$ at about 20 ms, which lies well above the CES = 0.60 contour, so extra compute is clearly worthwhile there. Figure 6 breaks the resolution effect down: quality peaks near the training resolution ($r = 112$–128) and declines beyond it, while time grows roughly with r^2; the fine-tuned model is better than stage 1 at every resolution.

The decision threshold has little influence (Figure 7). On the stage-1 model, τ between 0.40 and 0.55 changes Q by under 0.005 and $\tau = 0.6$ is worse; for the fine-tuned model 0.5 is equal or better than 0.45 at three of the four resolutions. We therefore keep the default $\tau = 0.5$ rather than tune it. Finally, Figure 8 checks robustness on 128 validation photographs with synthetic darkening, brightening and contrast reduction: the fine-tuned model is better by 0.018–0.024 in Q in all three conditions, consistent with the colour augmentation used in stage 2.

9 Results

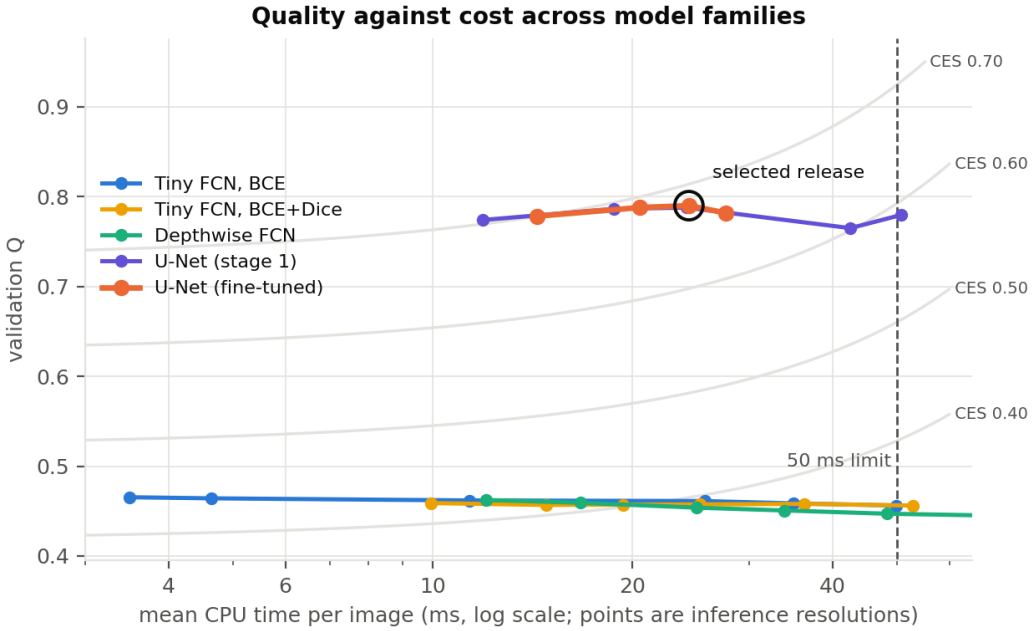


Figure 5. Validation Q against mean CPU time for every candidate. Each line connects one model family evaluated at different inference resolutions. Grey curves are lines of constant CES. The U-Net families sit far above the tiny networks, and the fine-tuned model (orange) is above its stage-1 predecessor at almost every speed.

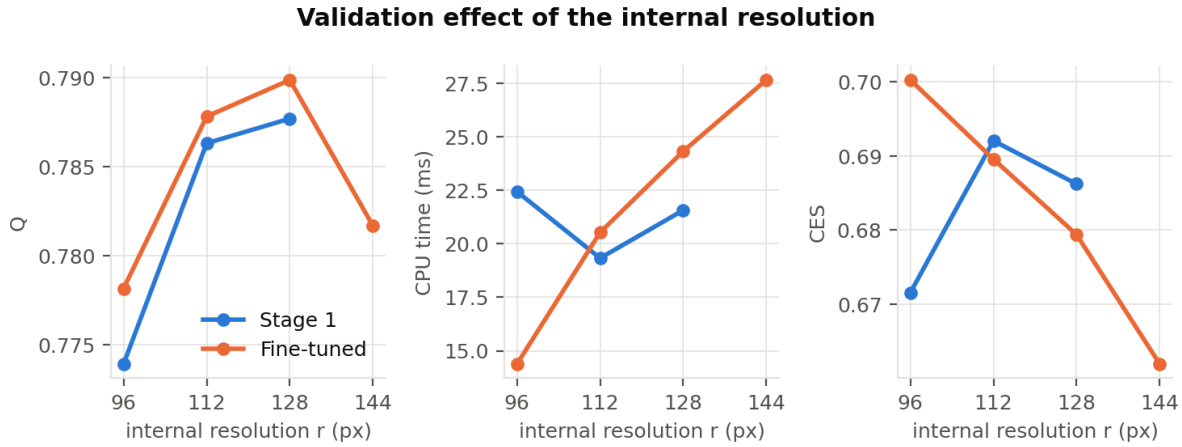


Figure 6. Validation Q, CPU time and CES against internal resolution for the stage-1 and fine-tuned U-Net. Timings come from one noisy session.

Set	mIoU	BF1	Q	CPU ms	Bytes	CES	N
Validation	0.8591	0.6284	0.7899	24.319	1787043	0.6794	1055
DUTS-TE	0.7144	0.4441	0.6333	17.141	1787043	0.5629	5019
Captured photos	0.6226	0.3238	0.5329	14.063	1787043	0.4802	10

Table 2. Final results of the single frozen model. Time is measured per set with `time.process_time` on one thread; DUTS-TE is the official number. Captured photos are scored against the supplied masks in `captured/masks_256` (see the note in the next section).

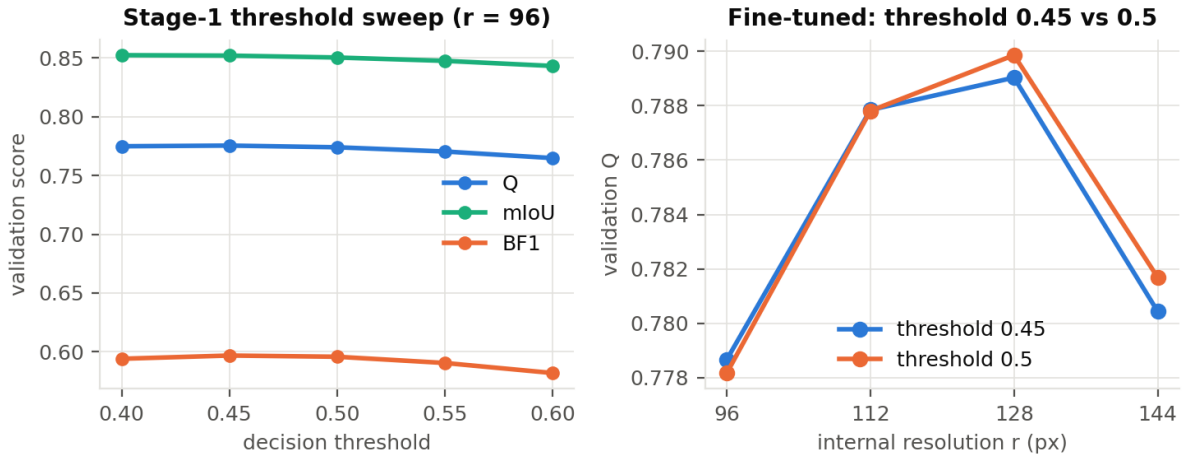


Figure 7. Left: stage-1 validation scores against the decision threshold. Right: fine-tuned validation Q at thresholds 0.45 and 0.5 for four resolutions.

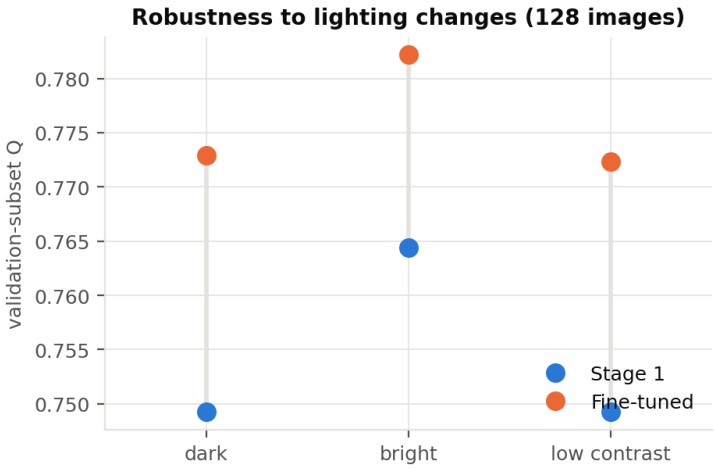


Figure 8. Validation-subset Q under synthetic lighting changes for the stage-1 and fine-tuned models.

The test CPU time is 17.1 ms, well inside the 50 ms limit, and the file is 1,787,043 bytes (0.43 % of 4 MiB), so the penalty terms cost about 11% of Q. Compared with the stage-1 model (DUTS-TE: mIoU 0.705, BF1 0.394, $Q = 0.612$, CES 0.544, 16.9 ms) fine-tuning raises Q by 0.022 and BF1 by 0.050 at nearly unchanged time. The gap between validation ($Q = 0.790$) and DUTS-TE ($Q = 0.633$) is large. Validation images come from the training distribution, whereas DUTS-TE contains many photographs with several objects, small objects and clutter, and the boundary score drops most (BF1 0.63 → 0.44): a 439k-parameter network cannot localize fine contours. CPU: Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42.

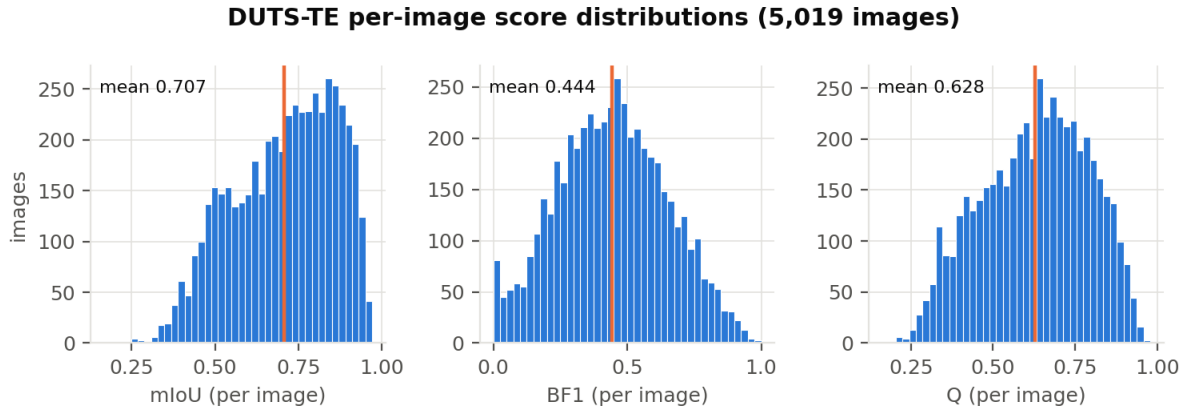


Figure 9. Per-image mIoU, BF1 and Q of the selected model on the 5,019 DUTS-TE photographs; the orange line marks the mean.

The distributions in Figure 9 show that the average hides a wide spread: per-image Q has a median of 0.64, a 10th percentile of 0.40 and a 90th percentile of 0.84; 16% of the photos reach $Q \geq 0.8$ while 10% fall below 0.4. The BF1 histogram has a spike at zero (1.4% of the images have no matched boundary pixels at all), typically photos where the network misses the object or splits it into fragments far from the true outline. Overlap and boundary quality are strongly related (Figure 10, correlation 0.85), but boundary accuracy is systematically lower: a ±2 pixel tolerance at 256 × 256 is strict for the soft edges produced by a model that works at 128 × 128.

Figure 11 shows where the lost quality goes. On every set the main error is a *missed object* (false negatives) rather than hallucinated foreground: object recall drops sharply from validation to the harder test photographs, whereas the background is classified correctly 87–96% of the time. This matches the failure cases below, where the network latches on to one salient part and leaves other object regions unlabelled.

10 Captured-photo results

Mask label: AI-generated mask.

Ten phone photographs (indoor and outdoor, near and far, cluttered scenes, a plant in a garden, bicycles in a shed) are stored in `captured/original` and resized to 256 × 256 in `captured/images_256`. The binary AI-generated masks in `captured/masks_256` are used as evaluation references. **Note:** these masks were produced with a pretrained promptable segmenter (EfficientSAM-Ti) and prompts chosen by an assistant, not drawn manually; they are used here only for evaluation, never for training or selection, and the numbers below inherit any errors in those masks. They are computed with the same metric code as the other sets, aggregating pixels over the ten photos.

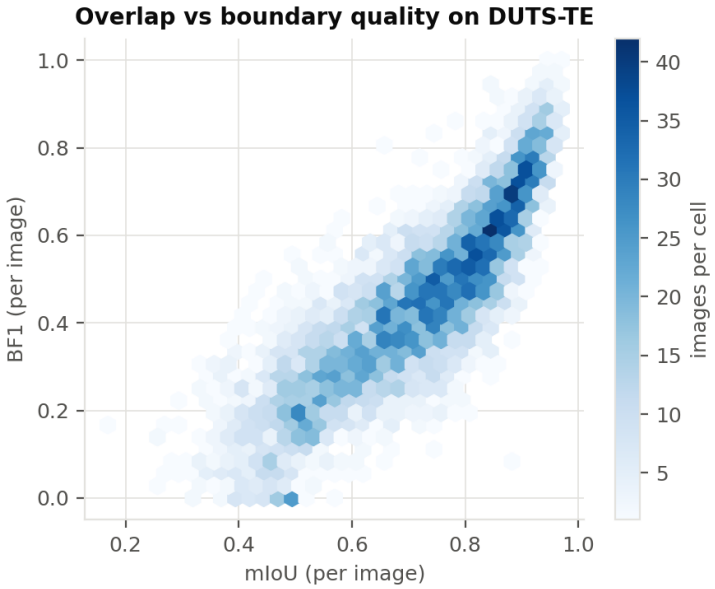


Figure 10. Per-image mIoU against BF1 on DUTS-TE (darker cells contain more images).

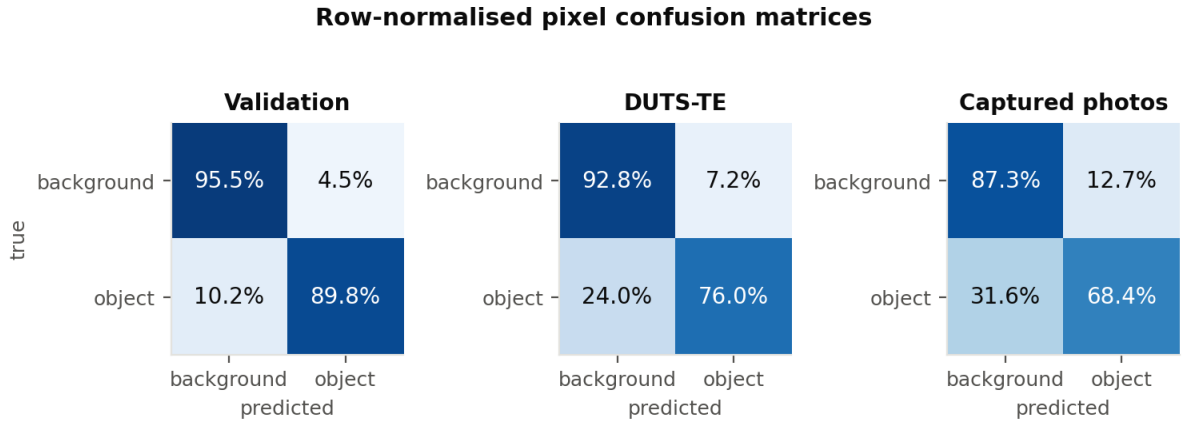


Figure 11. Row-normalised pixel confusion matrices of the selected model. Object recall falls from 89.8% on validation to 76.0% on DUTS-TE and 68.4% on our photographs, while background errors stay between 4.5% and 12.7%.

The fine-tuned model reaches mIoU 0.623, BF1 0.324, $Q = 0.533$ and CES 0.480 at 14.1 ms (Table 2). The stage-1 model scores mIoU 0.585, BF1 0.219, $Q = 0.475$, so fine-tuning helps here as well, but ten photos are far too few for a precise estimate. Table 3 gives per-photo values, and Figures 13 and 14 show the photograph, AI-generated mask, prediction and the two overlays for every photo.

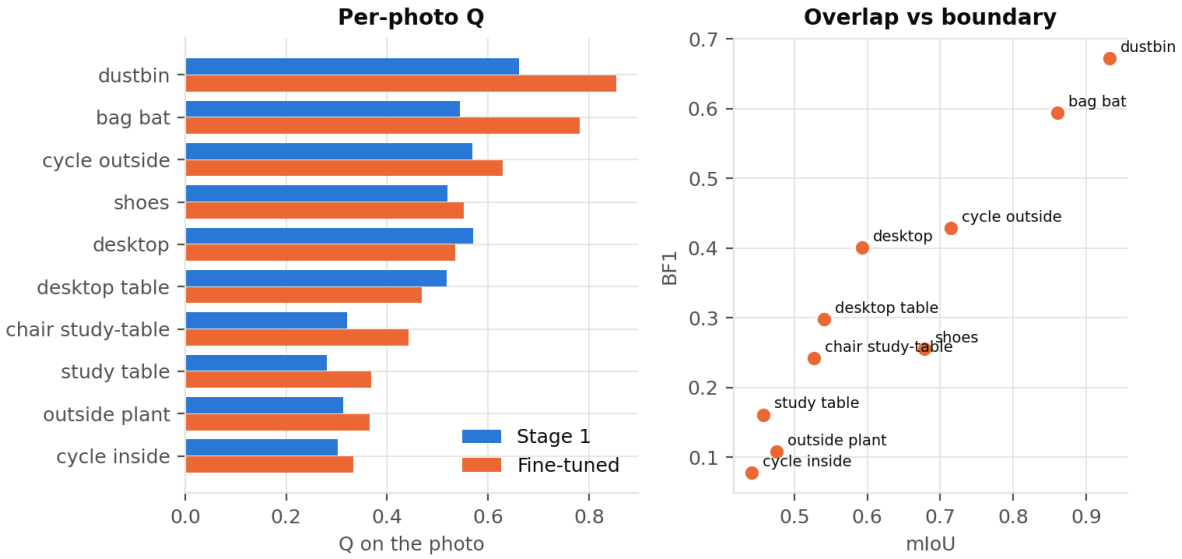


Figure 12. Left: per-photo Q for the stage-1 and fine-tuned model, sorted by the fine-tuned score. Right: per-photo mIoU against BF1 for the fine-tuned model.

Figure 12 shows that fine-tuning improves eight of the ten photos (the largest gains are on `bag_bat` and `dustbin`, which contain isolated, high-contrast objects) but is slightly worse on `desktop` and `desktop_table`, where the AI-generated mask covers a large piece of furniture rather than a compact salient item. The right panel again shows the tight relation between overlap and boundary quality, with the cluttered scenes in the lower left.

Photo	mIoU	BF1	Q
dustbin	0.932	0.672	0.854
bag_bat	0.861	0.594	0.781
cycle_outside	0.714	0.428	0.629
shoes	0.678	0.255	0.551
desktop	0.593	0.401	0.535
desktop_table	0.541	0.298	0.468
chair_study-table	0.527	0.242	0.442
study_table	0.458	0.161	0.369
outside_plant	0.476	0.108	0.365
cycle_inside	0.442	0.078	0.333

Table 3. Per-photo metrics of the selected model (each photo scored alone, so these do not average to the pooled values).

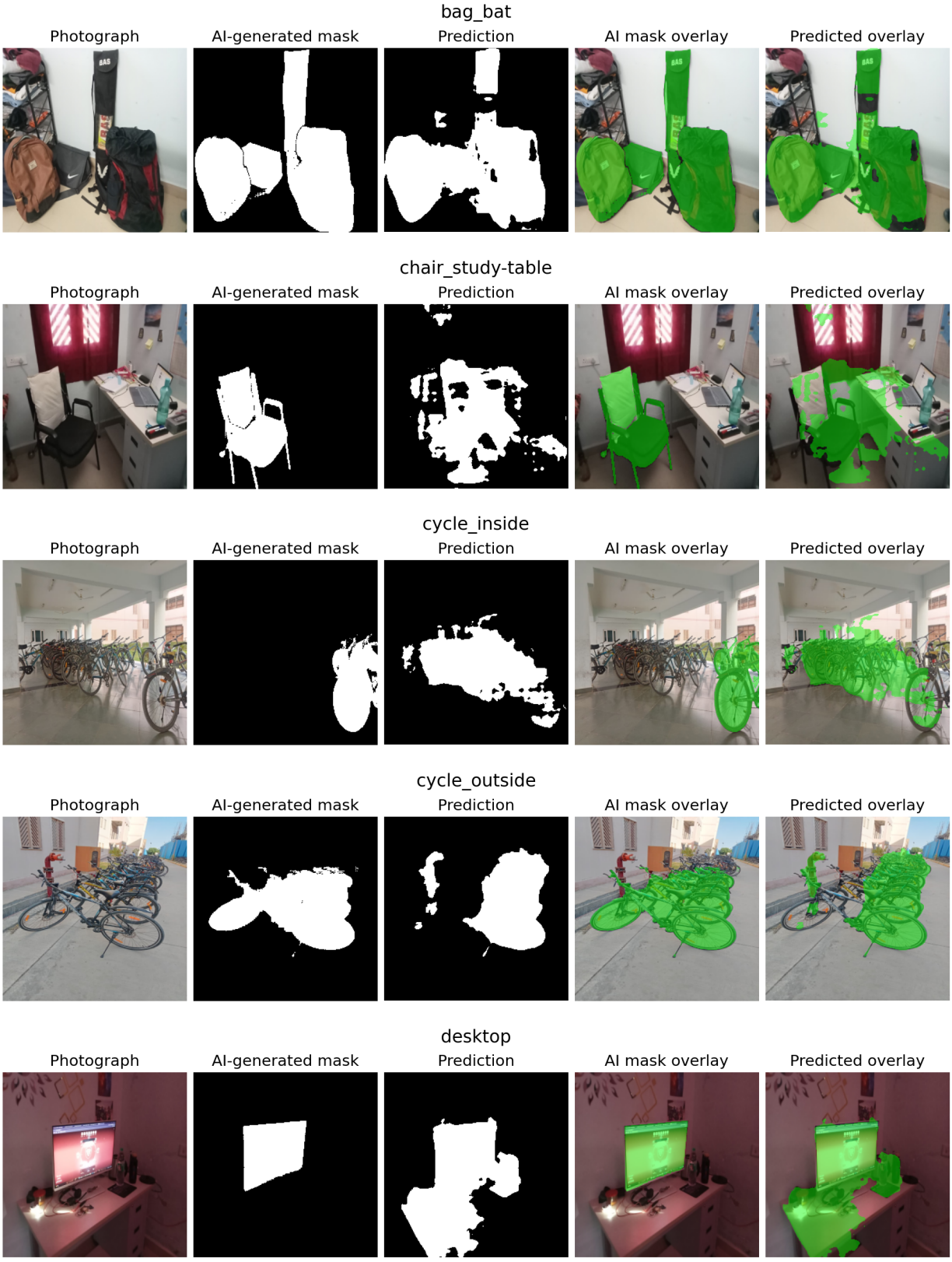


Figure 13. AI mask and predicted overlays, photos 1–5 (bag_bat, chair_study-table, cycle_inside, cycle_outside, desktop).

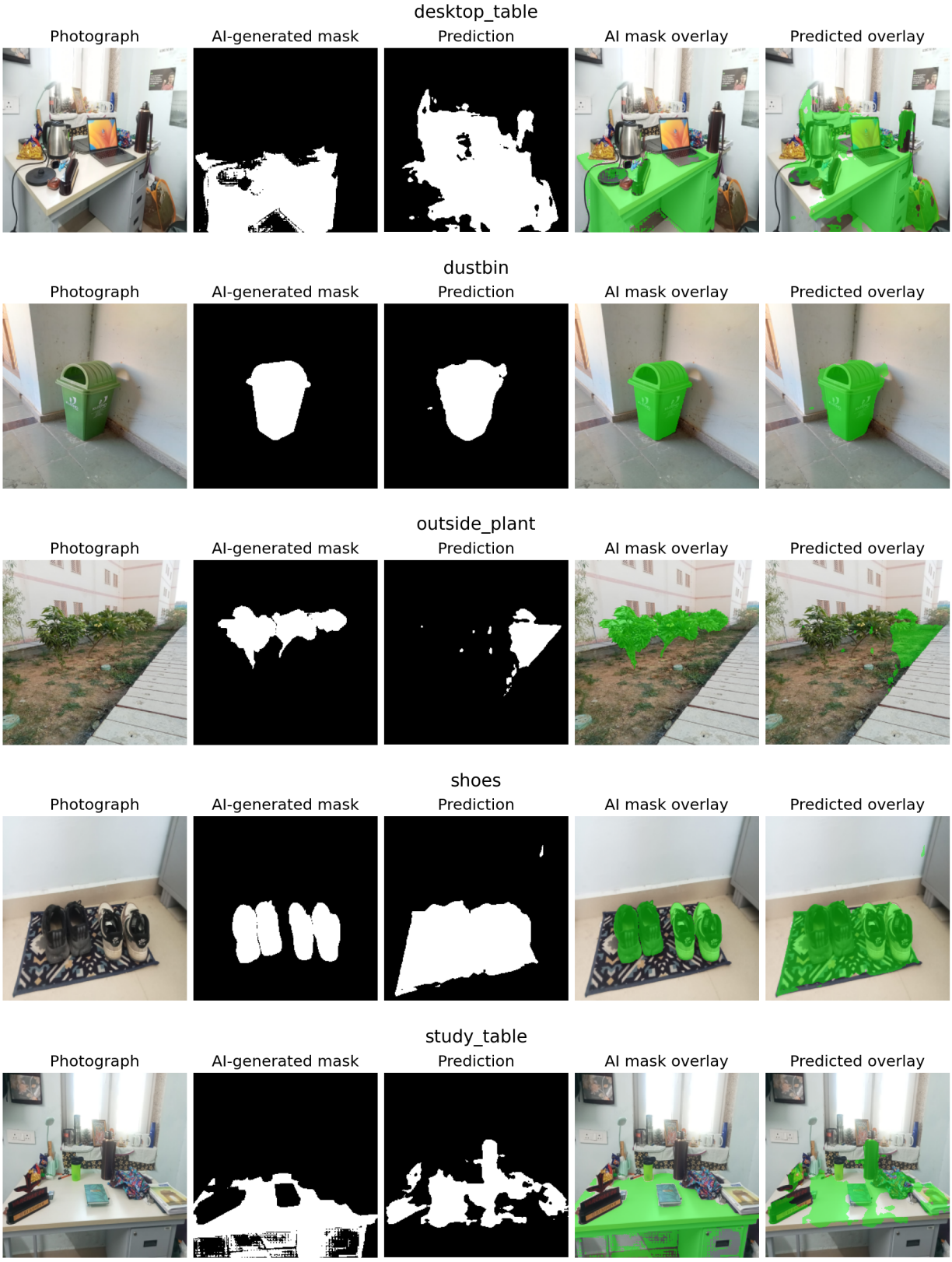


Figure 14. AI mask and predicted overlays, photos 6–10 (desktop_table, dustbin, outside_plant, shoes, study_table).

11 Failure analysis

The two lowest-scoring photos in Table 3 are the two failure cases (Figures 13 and 14).

- **Clutter and many similar objects** (`cycle_inside`, $Q = 0.333$, BF1 0.078). The AI-generated mask marks only the large bicycle in the right foreground, but the shed contains a whole row of bicycles. The network instead fires on the dense group of bicycles in the background and largely misses the foreground one. The task has no notion of "the object the user means"; DUTS teaches saliency as a high-contrast, centred object, and with many equal candidates a small network with little global context picks the textured cluster. The same pattern lowers `study_table` ($Q = 0.369$) and `chair_study-table`.
- **Low-contrast, irregular object** (`outside_plant`, $Q = 0.365$, BF1 0.108). The AI-generated mask marks the row of shrubs, which resemble the soil and weeds in colour and texture. The prediction misses the shrubs and instead selects the bright concrete footpath. Foliage boundaries are thin and ragged, so even where overlap is acceptable the contour score is very low.

A milder issue is bleeding onto the supporting surface or neighbouring structure (`shoes`, `dustbin`: a small spur near the wall), probably because object and support share edges and colour statistics. The best case is the isolated, high-contrast `dustbin` ($Q = 0.854$). Because the evaluation masks are AI-generated, part of the remaining error in any photo may come from the reference rather than the model.

12 Conclusion and limitations

A 439k-parameter U-Net trained from scratch and fine-tuned with BCE+Dice gives $Q = 0.633$ and CES $= 0.563$ on DUTS-TE at 17.1 ms and 1.79 MB, comfortably inside the compute budget, and $Q = 0.533$ on our ten own photographs. The score is limited by boundary accuracy and cluttered scenes, where the intended object is ambiguous for an automatic method. A larger model or higher resolution could raise Q while staying under 50 ms; we did not explore this because training is slow on a CPU. Limitations: a single training run per stage, the fine-tuning changes were not ablated one by one, no GPU, and the captured-photo evaluation references consist of AI-generated masks rather than hand-drawn annotations, so those numbers are indicative only.

13 Reproducibility

Run these PowerShell commands in an outputs-empty copy containing source, configurations and genuine input data. Existing submitted weights/results should be preserved in a separate copy. Backticks continue a command on the next line. The complete selected configuration is `configs/final.json`; the training command below reproduces it without touching either test set (the training configuration is `configs/final_train.json`). Evaluation and reproduction commands are documented in the root `README.md`.

```
python -m pip install -r requirements.txt
python -m scripts.download_duts --split TR
python -m scripts.prepare_dataset --root data/DUTS
python train.py --config configs/final_train.json --out experiments/final/unet16_bce_128
# Stage 2: BCE+Dice fine-tuning, then qualification and activation:
python train.py --config configs/quality_finetune.json --out experiments/quality/reproduction
python -m scripts.qualify_model --runs experiments/quality/reproduction `
  --out results/quality_new_run --activate
# Reuses that completed run, evaluates validation and records selection:
python -m scripts.run_experiments --config configs/final.json `
  --experiments configs/reproduce_experiments.json --out experiments/reproduce
python -m scripts.freeze_model
$tr = 'data/DUTS/DUTS-TR'
python evaluate.py --split validation --images "$tr/DUTS-TR-Image" `
  --masks "$tr/DUTS-TR-Mask" --manifest data/splits/val.txt `
  --out results/validation_metrics.json
python -m scripts.download_duts --split TE
$te = 'data/DUTS/DUTS-TE'
# This is both official quality evaluation and CPU timing:
python benchmark.py --split duts-test --images "$te/DUTS-TE-Image" `
  --masks "$te/DUTS-TE-Mask" --out results/duts_test_metrics.json
python predict.py --image photo.jpg --out mask.png
# Strict manual-annotation check (currently fails for AI-generated masks):
python -m scripts.verify_dataset --root captured --split captured
# Agreement evaluation against AI-generated masks:
python evaluate.py --split synthetic --images captured/images_256 `
  --masks captured/masks_256 --out results/captured_reference_metrics.json
python -m scripts.create_figures overlays --weights <active model.pth> `
  --images captured/images_256 --masks captured/masks_256 `
  --out results/captured_figures
python -m scripts.create_figures comparison
Push-Location report
pdflatex main.tex
pdflatex main.tex
Pop-Location
```

Prediction loads the active release named in `weights/active_model.json` (the stage-1 `weights/final_model.pth` is kept as the baseline) and prints exactly `T_CPU_ms:` and `S_disk_bytes:`. The first pass writes the mask; the next 20 passes are timed. Report building supports pdflatex or an explicitly supplied portable Tectonic executable. Every compiled page requires visual review. The strict `captured` split additionally requires a capture-metadata attestation; the evaluation against AI-generated masks above therefore uses the `synthetic` split and is reported as such.

References

[1] CSL7360, *Assignment 2: Cost-Efficient Segmentation under a Compute Budget*, supplied assignment PDF, pp. 1–3.

[2] L. Wang et al., *Learning to Detect Salient Objects with Image-level Supervision*, CVPR, 2017. Dataset: https://saliencydetection.net/duts/.